

16 Sampling, Continuous to Digital

The discrete time Fourier series is the only useful one in practice. When we sample data, it is inherently discrete. With an aperiodic signal, the definition goes from -infinity to +infinity, and we can't do that. The discrete series has a finite definition which we could compute in real life.

The only one with real data is the DFS. We take a chunk of data, pretend it is one period, and chug ahead. The other three tools are useful in theory. If we want to design a communication system, for example, we will need to use those. But for interpreting data read from a sensor, we need the discrete Fourier series.

Aside

This is a transform that represents the exact same thing in different forms. Similar to the transform pairs from polar to rectangular coordinates - just different representations that have a linear map.

$$re^{j\theta} = \sqrt{x_i^2 + x_r^2} e^{j \tan^{-1}(\frac{x_i}{x_r})}$$
$$x_r + jx_i = r \cos \theta + jr \sin \theta$$

Imagine we called these the polar transform and inverse polar transform. If all you knew was polar form, it would be hard to add complex numbers. But if we knew the rectangular form, that would be easy. The point is - having two equivalent representations that are different allows you to do far more complex operations.

We are taking a whole signal and representing it in a different way, and then manipulating it.

The Fast Fourier Transform

This is an efficient Implementation of the Discrete Fourier Transform (for some reason the exact same thing as the Discrete Fourier Series - why the transform vs series naming mix?).

The information that you are looking for is hard to see in the time domain, but very clear in frequency.

In speech recognition, we look at the spectrogram of a voice to recognize things (which is the same as how our brains do it - different frequencies travel different depths in our ears, and we have receptors at each depth).

Looking at radio, you'll have a terrible mess in time but neatly sectioned off bands in frequency.

Audio and image compression relies on Fourier compression of pixel data.

Take a sequence of data, take the DFT, cut out frequencies that you don't care about - and you have a much smaller representation. Its a little grainy when you go back to time, but still contains a lot of the data quality.

Sampling

Why do we need sampling?

If we replace a continuous time analog circuit with a digital system that has analog to digital and digital to analog converters, then we'll be able to do more manipulations with the data - logging, controlling, updating system behavior without manually replacing op amps, etc.

In the time domain, sampling a signal is just saying

$$x[n] = x(nT)$$

In the frequency domain, however, this is harder.

What we are really doing is making a delta function pulse train at $0, T, 2T$. We are multiplying this by $x(t)$ to get a continuous time signal but that only has the values at multiples of T .

$$\begin{aligned}
 p(t) &= \sum_{k=-\infty}^{\infty} \delta(t - kT) \\
 x_P(t) &= x(t)p(t) \\
 &= \sum_k x(t)\delta(t - kT) \\
 &= \sum_k x(kT)\delta(t - lT)
 \end{aligned}$$

We can write $x[n]$ as either a scalar or as a signal,

$$\begin{aligned}
 x[n] &= x(nT) \\
 &= \sum_k \delta(n - k)
 \end{aligned}$$

Lets do these steps in Fourier now.

We have an impulse train $p(t)$, lets get the frequency domain for that.

Lets take the Fourier series.

We find a_k :

$$\begin{aligned}
 p(t) &= \sum_k a_k e^{jk\omega_0 t} \\
 a_k &= \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} (t) e^{-jk\omega_0 t} dt
 \end{aligned}$$

This is just a single impulse at zero.

$$a_k = \frac{1}{T} \int_{-\frac{T}{2}}^{\frac{T}{2}} \delta(t) e^{-jk\omega_0 t} dt$$

This just pulls out

$$\begin{aligned}
 a_k &= \frac{1}{T} e^{-jk\omega_0(0)} \\
 a_k &= 1
 \end{aligned}$$

We know that $e^{j\omega_0 t} 2\pi\delta(\omega - \omega_0)$,

so

$$\begin{aligned}
 &e^{j\frac{2\pi}{T}kt} 2\pi\delta\left(\omega - \frac{2\pi}{T}k\right) \\
 &\sum_k a_k e^{j\frac{2\pi}{T}kt} \sum_k 2\pi a_k \delta\left(\omega - \frac{2\pi}{T}k\right) \\
 p(t) &= \sum_k \frac{1}{T} e^{j\frac{2\pi}{T}kt} \underbrace{\sum_k \frac{2\pi}{T} \delta\left(\omega - \frac{2\pi}{T}k\right)}_{P(j\omega)}
 \end{aligned}$$

At integer multiples of $\frac{2\pi}{T}$ we have a sequence of impulses, with magnitude $\frac{2\pi}{T}$.

We will now take this result and use it to see how sampling effects our signal.

In continuous time we have a sequence of impulses but whose magnitudes are the value of the function at that spot. This is the $x_p(t) = x(t)p(t)$.

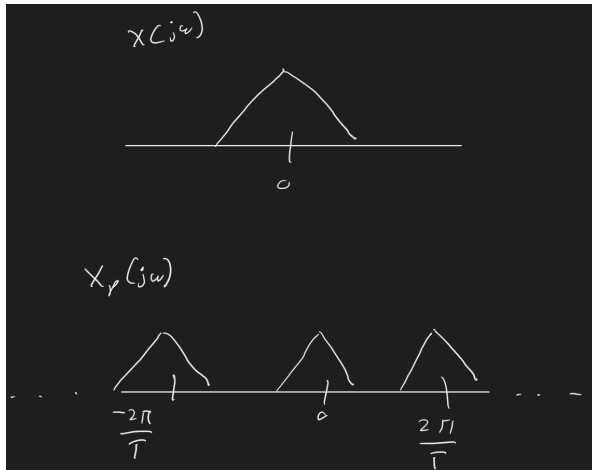
We know that $x(t)p(t) \xrightarrow{(j\omega)} P(j\omega) =_p(j\omega)$

We are defining the bandwidth of our generic $x(j\omega)$ as ω_0 .

$$\begin{aligned}
 x_p(j\omega) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} (j\theta) P(j(\omega - \theta)) d\theta \\
 &= \frac{1}{2\pi} \int_{-\infty}^{\infty} (j\omega) \cdot \frac{2\pi}{T} \sum_{k=-\infty}^{\infty} \delta\left(\omega - \theta - \frac{2\pi k}{T}\right) d\theta \\
 &= \frac{1}{T} \sum_k \int_{-\infty}^{\infty} (j\theta) \delta\left(\omega - \theta - \frac{2\pi k}{T}\right) d\theta
 \end{aligned}$$

$$p(j\omega) = \frac{1}{T} \sum_k \left(j \left(\omega - \frac{2\pi k}{T} \right) \right)$$

This is an infinite sum of scaled and shifted versions of our $(j\omega)$. Here I draw an arbitrary $x(j\omega)$ that doesn't actually matter, just for graphical depiction.



We could have found $p(j\omega)$ in a different way (we'll call this approach 2), that will also be a useful representation of the same thing.

We could say

$$x_p(t) = \sum_k x(kT) \delta(t - kT) \quad p(j\omega)$$

$$\begin{aligned}
 \delta(t - kT) &= \int_{-\infty}^{\infty} \delta(t - kT) e^{-j\omega t} dt \\
 &= e^{-j\omega kT}
 \end{aligned}$$

If we have a whole bunch of these impulses and scale them by constant scaling factors, it's just adding. Fourier analysis tools are just linear operators.

$$\sum_k x(kT) \delta(t - kT) = \underbrace{\sum_k x(kT) e^{-j\omega kT}}_{p(j\omega)}$$

This is the other useful way of writing the exact same $p(j\omega)$.

Now we can move to the very last step: converting from these sequence of impulses of different magnitudes in continuous time to a discrete time.

$$x[n] \quad (e^{j\hat{\omega}}) = ?$$

Lets compare the definition of the DTFT.

$$x_p(j\omega) = \sum_k x(kT)e^{-jk\omega T} \quad \text{from approach 2}$$

$$\begin{aligned} (e^{j\hat{\omega}}) &= \sum_k x[k]e^{-j\hat{\omega}k} && \text{definition of D} \\ &= \sum_k x(kT)e^{-j\hat{\omega}k} \end{aligned}$$

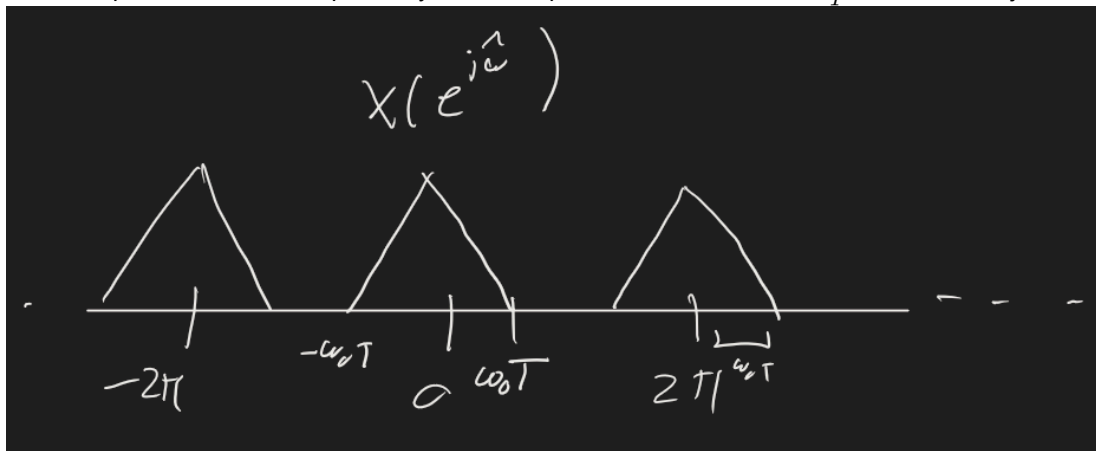
So $(e^{j\hat{\omega}})$ is just a frequency scaled version of $p(j\omega)$.

Since

$$p(j\omega) = \frac{1}{T} \sum_k \left(j \left(\omega - k \frac{2\pi}{T} \right) \right) \quad (\text{from approach 1})$$

$$\begin{aligned} (e^{j\hat{\omega}}) &= p \left(j \frac{\hat{\omega}}{T} \right) \\ &= \frac{1}{T} \sum_k \left(j \left(\frac{\hat{\omega} - 2\pi k}{T} \right) \right) \end{aligned}$$

This multiplies the location of peaks by T . So the peaks that used to be at $\frac{2\pi}{T}$ are moved to just 2π .



In reality, we can't make perfect impulse trains, so instead we take a zeroth order hold. This acts like a hold, so we say that data is constant for each sampling period until the next sample.

